Milestone 3 (M3) Report

Design and Implementation of an OFDM-Based Communication and Sensing Framework Using GNU Radio

GSoC category is open-source contributions to GNU Radio and SOTA positioning | Project G17 (individual)

Vamsi Pancholi (AU2440023) | GitHub vamsip-cell | Status as of 1 Nov 2026

Repository at https://github.com/vamsip-cell/g17_gsoc_isac_ind (folder M3_G17)

1. Summary of M3 Progress

M1 chose the problem and method, and M2 was about getting into the GNU Radio codebase. M3 reports the final status of the two upstream contributions made so far, with the evidence behind them, the feedback received, and the plan for the GSoC 2027 proposal. The main points are.

- **Upstream contributions.** Pull request #8231 (closes issue #7064) is open, its DCO check passes, and its CI workflows are waiting for maintainer approval. Nobody has reviewed it yet. On issue #7997, a community contributor suggested reproducing the freeze with a Channel Model, I replied on the issue on 1 Nov 2026 with the results of the M2 reproduction.
- **Findings on #7997.** The Channel Model does not cause the freeze, a stock OFDM chain with a channel works in all five tested variants, and two other configurations (a length-tag key collision and a forecast() mismatch) reproduce the freeze with or without a channel (Section 4.2).
- **Positioning and plan.** Section 4 lists the verification for each piece of work. Section 6 restates how the project differs from existing GNU Radio modules, and Section 8 is an initial draft of the GSoC 2027 proposal.

2. Final Contribution Status

Item	Status (as of 1 Nov 2026)	Link / location
PR #8231, rewrite Options block docs (closes #7064)	Open, 1 commit, DCO check passed, 3 CI workflows awaiting maintainer approval, 0 comments, no reviewers, no labels.	github.com/gnuradio/gnuradio/pull/8231
Issue #7997, OFDM flowgraph freeze with Channel Model	Investigated in M2 (script and results in the repo). Contributor feedback received, reproduction and findings posted in reply. Reporter has not replied yet.	github.com/gnuradio/gnuradio/issues/7997

The PR was written and submitted in M2, Sections 3.1 and 4.1 describe its content and verification. In summary, it replaces the outdated Options block documentation in grc/core/blocks/options.py, parameters are grouped by Generate Options mode, an incorrect description of Window Size is fixed, nine undocumented parameters are described, and a truncated sentence is repaired. Each description was checked against the workflow definitions and the generator code.

3. Implemented Changes and Technical Significance

3.1 Pull request #8231, Options block documentation (issue #7064)

The Options block documentation in grc/core/blocks/options.py was a leftover from before the workflow system and was no longer correct. The PR rewrites it, grouped by Generate Options mode (all modes, Advanced tab, GUI modes, No-GUI, Bokeh, QT, Hier Block, C++). It fixes four problems, mode-specific parameters were not marked with their mode, Window Size was described as the size of the flowgraph editor when it sets the size of the Bokeh web page (the generated code passes it to bokehgui.utils.run_server), nine visible parameters were undocumented (Thread-safe setters, Catch Block Exceptions, Hier Block Source Path, QSS Theme, Realtime Scheduling, Run Options, Run Command, Generate CMakeLists.txt, CMake options), and one sentence was cut off mid-word. Each description was checked against the workflow definitions (grc/blocks/*.workflow.yml) and the code that uses the parameter.

Technical significance: this text is the only in-tool explanation of the Options block that every GRC user sees. Two of the errors could mislead, for example a user changing Window Size expecting the editor to resize, or debugging without knowing that Catch Block Exceptions decides whether a crashing block thread is logged and stopped or terminates the program.

3.2 Issue #7997, findings on the OFDM freeze

The investigation shows that the reported freeze is not caused by the Channel Model itself. A stock OFDM transmitter and receiver run correctly with a Channel Model in between. The same total-freeze symptom does occur in two other configurations, a length-tag key collision between ofdm_tx and ofdm_rx (the receiver's internal header_payload_demux fails with “Buffer too small for min_noutput_items”), and a custom block whose general_work() waits for more input than its forecast() declares.

Technical significance: this narrows an unresolved report to two concrete, testable causes, and gives the reporter and maintainers a script to check them against. It is also directly relevant to the planned sensing blocks, which will sit next to ofdm_rx.

4. Testing and Validation Evidence

4.1 Pull request #8231, how the change was verified

- The patch applies cleanly to current main and GitHub reports that the PR merges cleanly.
- It passes the pycodestyle check that GNU Radio's CI runs (maximum line length 120, with CI's ignore list).
- GRC's own grc/tests/test_generator.py passes with the change.
- Open PR #8214 also edits options.py (it changes the visibility of the ID parameter). The two changes do not overlap, as noted in the PR description.
- CI status, the DCO check passes. The three CI workflows have not run because they await maintainer approval, so CI results are not yet available.

4.2 Issue #7997, reproduction results

A script (M2_G17/Code/src/repro_7997.py) builds each flowgraph and runs it in its own process with a 30-second timeout, so a frozen case cannot block the test. GNU Radio 3.10.9.2, Ubuntu 24.04. The stock cases use GNU Radio's ofdm_loopback parameters (64-point FFT, cyclic prefix 16, 50-byte packets) and run for 6 s, counting correctly decoded packets. The custom cases use a pilot-inserter-like gr.basic_block with a finite source and are OK if the flowgraph finishes. Full output is in M2_G17/Results/repro_7997_results_ubuntu.txt.

Case	What differs	Result
stock_no_channel	stock OFDM TX → RX, no channel	OK, 248/248 packets correct
stock_channel_eps1	Channel Model between TX and RX	OK, 248/248
stock_channel_eps1.0001	channel with timing offset 1.0001	OK, 247/247
stock_channel_tags_pass	channel passes tags	OK, 248/248
stock_channel_same_key	channel, receiver length-tag key set to the transmitter's key	OK, 247/247
stock_nochan_same_key	no channel, same length-tag key	**STALL**, Buffer too small for min_noutput_items
stock_chan_tagspass_same_key	channel passing tags, same key	**STALL**, same error
custom_no_channel	custom block, no channel	OK, 2000 in / 2000 out
custom_channel_eps1	custom block, Channel Model	OK, 1999 out (see note)
custom_channel_eps1.0001	custom block, channel with offset	OK, 1999 out
fragile_no_channel	block waits for 1000 vectors, forecast() says 1:1	**HANG**, no completion in 30 s
fragile_channel_eps1	same faulty block, with channel	**HANG**

Reading the table. All five stock cases with the Channel Model, including both timing offsets, work, so the channel is not the cause. The freeze appears only with the shared length-tag key (STALL) and with the forecast()/general_work() mismatch (HANG), and it appears with or without a channel. The one missing output vector in the custom channel cases is not a freeze, the Channel Model's fractional resampler holds back a few samples at the end of a finite stream, so the last OFDM symbol is incomplete. On macOS (GNU Radio from conda-forge), GNU Radio's ofdm_loopback.grc example, which contains a Channel Model, was also run in GRC and decoded packets normally.

Integration relevance. These two rules (use distinct length-tag keys, declare input needs correctly in forecast()) are design requirements for the sensing block, which will attach to the same OFDM receiver.

4.3 Limitations of this evidence

- **#7997** the reporter's exact flowgraph and block code are unavailable, so their freeze could not be reproduced directly, the stated status is “not reproducible with stock blocks or a correctly written custom block”. The script ran on GNU Radio 3.10.9.2, while the reporter used 3.10.11. All tests are simulation-only.
- **#8231** documentation correctness was checked against the code, not by maintainers, and the CI workflows have not run yet.

5. Maintainer and Community Feedback, and Response

5.1 Pull request #8231

As of 30 Oct 2026, three weeks after opening, the PR had no comments or reviews. The only status shown is that the DCO check passed and that three CI workflows are waiting for a maintainer to approve them, which GitHub requires for a first-time contributor's PR. I did not post a reminder, the PR stays open and I will respond to any review.

5.2 Issue #7997

About two weeks before this check, a contributor on the issue (ffu, labelled “Contributor”) commented that the freeze could be tested by placing the Channel Model between the OFDM transmit and receive blocks, and that pilot_inserter is a custom Python block about which there was not enough information, so the reporter should post their flowgraph. The reporter had not replied as of 30 Oct.

The M2 reproduction already covers what was suggested. My response: On 1 Nov 2026 I replied on the issue saying that I had tried reproducing the freeze on GNU Radio 3.10 on Ubuntu. A stock OFDM chain with a Channel Model in between ran without freezing, the freeze appeared only when the receiver's length-tag key matched the transmitter's (“Buffer too small for min_noutput_items”), or when a custom block's general_work() waited for more input than its forecast() declared. I asked the reporter to share their flowgraph and the pilot_inserter source so I can check whether one of these two cases applies. Whether either applies to the reporter's flowgraph cannot be decided until they respond.

5.3 What I learned from this

- A volunteer-reviewed project can take weeks to respond, so a contribution should be self-contained, the PR description lists what was checked against what, and the issue reply states concrete results from a reproducible test instead of guesses.
- The #7997 feedback is useful because it asks for the same thing the investigation needed, the reporter's code. Posting the reproduction turns an unresolved question into a specific request.

6. Revised SOTA Positioning

The positioning from M2 holds, with the new evidence added. The GNU Radio ecosystem already has OFDM communication and OFDM radar, but as separate tools.

Module	**OFDM data communication**	**Range / Doppler sensing**	**Both from the same frames**
gr-digital (in-tree)	Yes (ofdm_tx, ofdm_rx)	No	No
gr-radar (KIT)	No	Yes (CW, FSK, FMCW, OFDM radar toolbox)	No
gr-ofdmradar (Analog Devices)	No	Yes (OFDM radar TX/RX)	No
This project	Reuses gr-digital's chain	Reciprocal filtering + 2D FFT	**Planned**

This is based on the module documentation and code I read in M1 and M2. I have not found a GNU Radio implementation that decodes the data and produces a range-Doppler map from the same OFDM frames, but I cannot rule out that one exists outside the places I searched, so the claim is limited to “not found”. The upstream work adds two things to the earlier positioning.

- **A concrete interface requirement.** The sensing path needs the transmitted symbols aligned with the received frame, so the block design has to carry them (through tags, a message port or a second stream). This is the main design question for the proposal.
- **Known failure modes of the host chain.** From #7997, do not share length-tag keys with ofdm_tx, and declare forecast() consistently with general_work().

The method itself is not new, it follows Sturm and Wiesbeck (2011), Braun (2014) and the tutorial of Dai et al. (2025). What the project adds is the integration with GNU Radio's communication chain, tests, and documentation.

7. Remaining Work and Honest Status

- No GNU Radio sensing block exists yet. The main implementation work is still ahead, which is what the proposal plans.
- PR #8231 and issue #7997 are waiting on others. The PR may need rebasing if #8214 (which touches the same file) is merged first.
- Nothing has been tested with an SDR or with real channel effects.

8. Initial Draft of GSoC 2027 Proposal

8.1 Title and summary

OFDM-based joint communication and sensing blocks for GNU Radio

The project adds an out-of-tree module with blocks that take the frames of GNU Radio's OFDM receiver together with the known transmitted symbols, and produce a range-Doppler map, so that one OFDM waveform can carry data and measure range and velocity.

8.2 Topic background and problem

Integrated sensing and communication (ISAC) uses one waveform for both tasks, and OFDM is the main candidate. GNU Radio has OFDM communication (gr-digital) and OFDM radar (gr-radar, gr-ofdmradar) but, as far as I found, nothing that does both on the same frames (Section 6). Researchers and students who want to prototype ISAC on an SDR therefore have to write their own processing outside GNU Radio.

The theory behind the method is reciprocal filtering (dividing each received symbol by the known transmitted symbol) followed by a 2D FFT over subcarriers and OFDM symbols, so that delay appears as a phase slope across subcarriers and Doppler as a phase slope across symbols (Sturm and Wiesbeck 2011, Braun 2014, Dai et al. tutorial, arXiv 2504.19091). The existing code I have read is GNU Radio's gr-digital OFDM transmitter and receiver (ofdm_txrx.py and the blocks behind it), gr-

radar (KIT) and gr-ofdmradar (Analog Devices). The project is needed because none of them does data decoding and range-Doppler estimation on the same frames.

8.3 Proof of capability (prior work)

- PR #8231 (Options block documentation), submitted and awaiting review.
- Investigation of #7997 with a reproducible script and findings that apply directly to the design of the sensing blocks.
- Profile and repository at https://github.com/vamsip-cell and https://github.com/vamsip-cell/g17_gsoc_isac_ind (folders M1_G17, M2_G17, M3_G17).

8.4 Deliverables

1. **Reciprocal-filter block** takes a received OFDM frame vector and the aligned transmitted symbol vector and outputs their element-wise quotient, with correct tag propagation.
2. **Range-Doppler block** collects P filtered symbols, applies the IFFT over subcarriers and FFT over symbols with optional zero-padding, and outputs the magnitude map. A peak detector (a simple threshold first, CFAR as a stretch goal) outputs range and velocity.
3. **Interface to ofdm_rx** a documented way to hand the transmitted symbols and frame boundaries to the sensing path, without sharing length-tag keys with the transmitter (the lesson from #7997).
4. **Example flowgraphs** a simulated-target loopback that decodes data and shows the range-Doppler map from the same frames, and, if hardware is available, an SDR version.
5. **Tests** unit tests with gr_unittest against a NumPy reference (a simple reference implementation of the same processing), run in CI.
6. **Documentation** GRC block definitions with parameter descriptions, a README, and a short tutorial.

Platform and language host only (no USRP-specific code), C++ for the signal-processing blocks with Python QA and examples, unless mentors prefer Python for the first version (see questions). **Out of scope** angle estimation and MIMO, multi-target tracking, hardware-specific tuning, and any changes to gr-digital's existing OFDM receiver unless agreed with mentors. **License** GPLv3, as GNU Radio requires.

8.5 Schedule (12 weeks, within the 10–22 week range)

Documentation and QA are written alongside each block, not left to the end.

Weeks	Work	Outcome
Community bonding	Discuss design with mentors, set up the OOT module (gr_modtool), agree on the interface and data types	Agreed design note
1–2	Reciprocal-filter block (C++ or Python, as agreed), unit tests against NumPy, block documentation started	Block + tests + docs stub
3–4	Range-Doppler block with zero-padding and map output, test with simulated targets	Block matches the NumPy reference
5	Peak detection (threshold), range and velocity outputs, **midterm, loopback flowgraph with simulated target**	Working simulated demo
6–7	Interface to ofdm_rx, transmitted symbols, frame alignment, tags, handle forecast and tag rules	Data + sensing from the same frames
8–9	Example flowgraphs, GRC blocks, Doppler/CP/ICI effects in	Examples + extended tests

	tests, clutter and direct-path leakage in simulation	
10	SDR experiment if hardware access is available (otherwise recorded-IQ test)	Hardware or file-based result
11	Finish tutorial and documentation (written alongside each block from week 1), CI, performance check, bug fixing	Docs complete
12	Buffer, review feedback, final report and packaging	Final submission

8.6 Testing and validation

- NumPy reference for every block, compared sample by sample.
- Accuracy test is range and velocity error against SNR for simulated targets, compared with a baseline that skips reciprocal filtering, run on the GNU Radio blocks.
- Stress tests for scheduler issues of the kind found in #7997 (separate process, timeout).

8.7 Risks

- **Synchronisation and alignment.** Getting the transmitted symbols aligned with the received frame in a real chain is the hardest part. Mitigation, start with loopback where alignment is exact, then add offsets step by step.
- **Hardware access.** A USRP B210 is available in my university lab, so an SDR experiment is planned for week 10, subject to lab access. If it is not usable in time, the project is still complete with simulation and recorded data.
- **Real-channel effects** (direct-path leakage, clutter) may limit accuracy and are listed as extensions rather than promised results.

8.8 Questions for mentors

- Should this be an out-of-tree module, or is there interest in part of it in gr-digital?
- What is the preferred way to pass the transmitted symbols to the sensing path, using stream tags, a second stream, or a message port?
- Is gr-radar's existing OFDM code a better base to extend than a new module?
- Is C++ expected for the blocks, or is a Python implementation acceptable for the first version?
- Are there SDR devices or recorded OFDM captures the project can use for testing?

8.9 Time commitments

- Project size is 175 hours, which is about 15 hours per week over 12 weeks.
- Weekly coding hours are about 15. I have no planned time off. I have no other summer 2027 commitments planned.

8.10 About me and formalities

- Name: Vamsi Pancholi. Time zone is IST (UTC+5:30). University and student status: Ahmedabad University, enrolled B.Tech student in Computer Science and Engineering. Residence: India. The project is part of a course (tied to university work).
- I am a B.Tech student in computer science and engineering, and my research interest is integrated sensing and communication. Languages I know well are Python, C++ and C.
- Open-source experience is none before this project. My contributions so far are PR #8231 and the #7997 investigation (Section 2).

- Communication plan is weekly public updates (blog or mailing list), regular check-ins with my mentor by email and on the GNU Radio mailing list, and questions on the mailing list and chat.
- English proficiency is fluent. I have just started contributing to GNU Radio and plan to keep fixing issues and maintaining the module I build after GSoC.

8.11 Statements required by the GNU Radio page

- I have read and understood the three strikes rule and acknowledge it.
- Cyberspectrum is the best spectrum.

References

[1] C. Sturm and W. Wiesbeck, "Waveform Design and Signal Processing Aspects for Fusion of Wireless Communications and Radar Sensing," Proc. IEEE, vol. 99, no. 7, pp. 1236–1259, 2011, doi 10.1109/JPROC.2011.2131110.

[2] M. Braun, "OFDM Radar Algorithms in Mobile Communication Networks," Ph.D. dissertation, Karlsruhe Institute of Technology, 2014, doi 10.5445/IR/1000038892.

[3] M. Mirabella, P. D. Viesti, A. Davoli, and G. M. Vitetta, "Deterministic Signal Processing Techniques for OFDM-Based Radar Sensing, An Overview," IEEE Access, vol. 11, pp. 68872–68889, 2023, doi 10.1109/ACCESS.2023.3292937.

[4] Q. Dai, Y. Zeng, H. Wang, C. You, C. Zhou, H. Cheng, X. Xu, S. Jin, A. L. Swindlehurst, Y. C. Eldar, R. Schober, R. Zhang, and X. You, "A Tutorial on MIMO-OFDM ISAC, From Far-Field to Near-Field," arXiv:2504.19091v2, 2025.

[5] gr-radar, GNU Radio Radar Toolbox. https://github.com/kit-cel/gr-radar

[6] gr-ofdmradar (Analog Devices). https://github.com/analogdevicesinc/gr-ofdmradar

[7] GNU Radio, gr-digital/python/digital/ofdm_txrx.py. https://github.com/gnuradio/gnuradio/blob/main/gr-digital/python/digital/ofdm_txrx.py

[8] GNU Radio pull request #8231. https://github.com/gnuradio/gnuradio/pull/8231

[9] GNU Radio issue #7064. https://github.com/gnuradio/gnuradio/issues/7064

[10] GNU Radio issue #7997. https://github.com/gnuradio/gnuradio/issues/7997

[11] Project repository. https://github.com/vamsip-cell/g17_gsoc_isac_ind (folders M2_G17 and M3_G17)